Laboratorium 6 - Cross-Site Scripting

3 zadania po 1 pkt. 90 minut (2 × 45). Plan: 10 minut wprowadzenia, 20 + 25 + 20 minut na zadania, 15 minut na raport i oddanie. Na początku zajęć uruchom przygotowane stanowisko według `START.md`. Atakujesz i naprawiasz dostarczoną, celowo podatną aplikację „Tablica ogłoszeń” (Python, Flask). Ładunki i poprawki piszesz samodzielnie.

Dozwolone: ISOD, dokumentacja, własne notatki i własny wcześniej napisany kod. Podczas zajęć wyłącz czaty i podpowiedzi LLM. Przy odbiorze wyjaśniasz ataki i poprawki. Atakujesz wyłącznie lokalną aplikację z pakietu, pod adresem `http://127.0.0.1:8080`.

Uruchomienie i sprawdzanie

Wszystkie polecenia wykonuj w katalogu `pliki-dla-studentow`. Uruchom `docker compose up -d`. Kod aplikacji jest w `aplikacja/`; po każdej zmianie wykonaj `docker compose restart web`. Restart usuwa dodane ogłoszenia, a zalogowane sesje pozostają ważne. Logi aplikacji: `docker compose logs --tail 30 web`.

Konta ćwiczeniowe: `anna` z hasłem `anna-lab6` (ofiara) i `student` z hasłem `student-lab6`. Drugie konto otwieraj w oknie prywatnym przeglądarki. Zbieracz, czyli serwer napastnika, działa pod `http://localhost:8081` i zapisuje w logu każde żądanie: `docker compose logs zbieracz`. Ładunki mają być nieszkodliwe: zmieniają wygląd lub tytuł strony albo wysyłają dane wyłącznie do lokalnego zbieracza. W Windows używaj `curl.exe`.

Pomoc i zapis wyników

Pomoc podczas zajęć: `pliki-dla-studentow/PRZEWODNIK.md`. Raport ma jedną stronę: zwięzłe wyniki i wnioski. Długie polecenia, ładunki i logi możesz zapisać w `dowody.md` w repozytorium, wskazując sekcje w raporcie. Skracaj tokeny; nie zapisuj haseł ani kluczy prywatnych. Część dodatkowa nie jest wymagana na pełne 3 pkt.

Zadanie 1 - XSS odbity (20 min, 1 pkt)

1. Otwórz `/szukaj?q=konsultacje`. W `aplikacja/app.py` znajdź miejsce, w którym parametr `q` trafia do HTML.
2. Przygotuj adres `/szukaj?q=...`, którego otwarcie zmienia tło strony na czerwone i tytuł karty na `XSS-odbity`.
3. Popraw kod tak, aby wartość `q` była kodowana jako tekst HTML. Użyj funkcji `escape` z modułu `html`, już zaimportowanej w aplikacji. Nie usuwaj wyszukiwarki i nie filtruj pojedynczych słów, np. `script`.
4. Po restarcie otwórz ten sam adres. Ładunek ma się wyświetlić jako zwykły tekst, a wyszukiwanie `konsultacje` ma nadal znajdować ogłoszenie.

W raporcie: adres z ładunkiem, numer wiersza z błędem, fragment HTML odpowiedzi przed poprawką i po niej oraz jedno zdanie: dlaczego kodowanie wyjścia jest lepsze od usuwania słowa `script`.

Zadanie 2 - XSS zapisany i przejęcie sesji (25 min, 1 pkt)

1. Zaloguj się jako `student` i dodaj ogłoszenie z ładunkiem, który wysyła `document.cookie` do zbieracza, np. jako parametr adresu obrazka `http://localhost:8081/?c=...`. Ładunek ma działać u każdej osoby, która otworzy tablicę.
2. W oknie prywatnym zaloguj się jako `anna` i otwórz tablicę. W logu zbieracza odszukaj ciasteczko sesji Anny.
3. Użyj skradzionej wartości bez znajomości hasła: `curl -i -b "session=WARTOŚĆ" http://127.0.0.1:8080/notatka`. Zapisz, czyja notatka została zwrócona.
4. Znajdź w `app.py` ustawienie, które pozwala skryptom odczytać ciasteczko sesji, i zmień je. Po restarcie wyloguj się i zaloguj ponownie na obu kontach, aby przeglądarka dostała nowe ciasteczka z HttpOnly. Następnie ponownie dodaj ogłoszenie i otwórz tablicę Anny. Sam restart nie zmienia atrybutów już zapisanego ciasteczka ani nie unieważnia skradzionego tokenu. Porównaj nagłówek `Set-Cookie` z

odpowiedzi na logowanie przed zmianą i po niej: `curl -i -d "login=anna&haslo=anna-lab6" http://127.0.0.1:8080/logowanie`.

W raporcie: treść ładunku, wpis zbieracza przed zmianą i po niej (wartość ciasteczka skróć do kilkunastu znaków), wynik kroku 3, oba nagłówki `Set-Cookie` ze skróconą wartością tokenu oraz jedno zdanie: dlaczego HttpOnly nie usuwa podatności XSS.

Zadanie 3 - Naprawa i Content Security Policy (20 min, 1 pkt)

1. W szablonach w `aplikacja/templates/` znajdź miejsce, które wyłącza automatyczne kodowanie treści ogłoszenia, i przywróć kodowanie.
2. Strona `/podglad` wstawia tekst po znaku `#` przez `innerHTML`. Przygotuj adres `/podglad#...` z ładunkiem w atrybucie zdarzenia, np. `onerror`, który zmienia tytuł karty na `XSS-DOM`. Następnie przenieś skrypt strony do pliku `aplikacja/static/podglad.js` i użyj `textContent`.
3. Dodaj do każdej odpowiedzi nagłówek `Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'; base-uri 'self'`. Użyj dekoratora `@app.after_request`.
4. Po restarcie sprawdź trzy ładunki: odbity, zapisany i DOM. Dodaj też ogłoszenie `Zażółć gęślą jaźń: <b>pogrubienie</b> & "cudzysłów" 'apostrof'` i sprawdź, czy wyświetla się dosłownie. Podgląd ma nadal pokazywać zwykły tekst.

W raporcie: tabela trzech ładunków z wynikiem przed naprawą i po niej, nagłówek CSP z odpowiedzi, wynik testu tekstu oraz jedno zdanie: dlaczego CSP jest dodatkową warstwą, a nie zamiennikiem kodowania wyjścia.

Dodatkowo - dla chętnych, poza zakresem 90 minut

Te ćwiczenia nie są wymagane do uzyskania pełnych 3 pkt ani do raportu.

- Tymczasowo przywróć wyłączone kodowanie przy włączonym CSP. Sprawdź w konsoli DevTools, które dyrektywy blokują skrypt inline i wysyłkę do zbieracza.
- Dopuść w ogłoszeniach wyłącznie znaczniki `<b>` oraz `<i>` biblioteką do sanityzacji HTML, np. `nh3`. Sprawdź, że `onerror` i `<script>` zostają usunięte.

Oddanie - jeden plik raportu do końca zajęć

Oddaj **jeden plik w formacie tekstowym**, np. **TXT, MD lub PDF z tekstem**. **Nazwa pliku jest dowolna. Raport należy oddać do końca zajęć** w miejscu wskazanym przez prowadzącego. Raport do jednej strony zawiera imię i nazwisko, numer indeksu, link do repozytorium GitHub, identyfikator commitu oraz wyniki i krótkie odpowiedzi z trzech zadań. Uzupełniaj go podczas pracy; oddaj również ukończone części.

W repozytorium umieść poprawioną aplikację z katalogu `aplikacja/` i README z uruchomieniem. Możesz dołączyć pliki środowiska z pakietu. Repozytorium musi być dostępne dla prowadzącego. Nie wpisuj pełnych wartości ciasteczek. Nie oddawaj ZIP-a, osobnych załączników z kodem ani samych zrzutów ekranu.

Punktacja

Za każde zadanie: **0,50 pkt** za działający własny ładunek i poprawkę w repozytorium, **0,25 pkt** za wymagane wyniki w raporcie i **0,25 pkt** za poprawne wyjaśnienie. Łącznie **3 pkt**. Instalacja środowiska nie jest osobnym zadaniem punktowanym.